

Trabajo Final Integrador

Materia: Análisis y Diseño de Algoritmos

Universidad EAFIT

Estudiantes: Athina Cappelletti, Juan Trujillo, Andrés Pérez

Índice

1. Introducción	1
2. Descripción del dataset	2
3. Módulo A - Divide y Vencerás	2
4. Módulo B - Codicioso	5
5. Módulo C - Programación Dinámica	8
6. Integración del pipeline	12
7. Conclusiones	12
8. Herramientas utilizadas	12
9. Referencias	13
10. Roles del equipo	13

1. Introducción

El proyecto está desarrollado con una solución algorítmica para un problema de tipo planificación de recursos en redes de telecomunicaciones. El objetivo principal y general es procesar solicitudes de servicio, construir una red de conexión de bajo costo y asignar ancho de banda limitado entre solicitudes activas. Para esto se utiliza el dataset Telecom Customer Churn, compuesto por 7,043 registros de clientes, donde cada cliente se interpreta como una solicitud de servicio con atributos como antigüedad, cargos mensuales, cargos acumulados y estado de churn.

Nuestra solución se divide en tres módulos. El primero, el módulo A el cual aplica Divide y vencerás mediante mergesort y búsqueda binaria recursiva, ya que se requiere ordenar eficientemente las solicitudes por prioridad y realizar consultar sobre el arreglo ordenado. El segundo, el módulo B el cual aplica un algoritmo codicioso, kruskal, para construir un árbol de expansión mínima sobre una topología de red generada de forma determinista, este paradigma es adecuado porque en los árboles de expansión mínima las decisiones locales válidas conducen a una solución óptima global. Y por último, el módulo C, el cual aplica programación dinámica para resolver el problema de la Mochila 0-1, donde se busca maximizar el valor de las solicitudes seleccionadas sin exceder una capacidad de ancho de banda.

La integración de los tres módulos permite construir un pipeline completo en C++: primero se cargan y ordenan los datos, luego se genera la red de mínimo costo y finalmente se asigna ancho de banda usando las solicitudes activas de mayor prioridad. De esta manera, el proyecto permite aplicar y comparar tres paradigmas fundamentales del análisis de algoritmos sobre datos reales y reproducibles.

2. Descripción del dataset

El archivo WA_Fn-UseC_-Telco-Customer-Churn.csv contiene 7,043 registros de clientes con 21 atributos. Al cargarlo se extraen mediante el parser, únicamente los campos: customerID, tenure, MonthlyCharges, TotalCharges y Churn.

Estadísticas del CSV cargado:

Propiedad	Valor
Total de registros cargados	7,043
Registros con TotalCharges nulo	11
Registros con Churn = No (activos)	5,174
tenure máximo	72
tenure mínimo	0
MonthlyCharges promedio global	64.76 USD

Manejo de valores nulos en TotalCharges

Los 11 registros con tenure = 0 tienen el campo TotalCharges como espacio en blanco (no como número). El parser detecta este caso verificando si el string contiene solo espacios, y asigna 0.0 en esos registros.

Construcción determinista del grafo (Módulo B)

Los 7,043 registros se distribuyen en $N = 20$ grupos asignando el registro i al grupo $i \bmod 20$. Para cada grupo k se calcula $\bar{M}_k$ = promedio de MonthlyCharges de ese grupo, redondeado a dos decimales. El costo de la arista entre los nodos u y v (con $u < v$) se define como $c(u,v) = [\bar{M}_u + \bar{M}_v]$. Se crean aristas para todos los pares, generando un grafo completo de 20 nodos y 190 aristas.

3. Módulo A - Divide y Vencerás

Para el ordenamiento de las 7,043 solicitudes se implementó el algoritmo MergeSort, una estrategia basada en el paradigma Divide y Vencerás que divide recursivamente el arreglo en mitades, ordena cada mitad de forma independiente y las combina en un arreglo ordenado final.

Se eligió MergeSort sobre QuickSort por dos razones fundamentales. Primero, MergeSort garantiza un comportamiento de $O(n \log n)$ en el peor caso, mientras que QuickSort degrada a $O(n^2)$ con pivotes desfavorables (por ejemplo, cuando el arreglo ya está ordenado). Segundo, MergeSort es un algoritmo estable: preserva el orden relativo de registros con el mismo valor de tenure, propiedad relevante cuando múltiples clientes comparten la misma antigüedad y se desea mantener consistencia en los resultados.

Para la implementación, nos basamos en un video que explica el funcionamiento del algoritmo MergeSort, del cual extrajimos el siguiente pseudocódigo como guía:

```
mergesort (array a)
    if ( n == 1 )
        return a

    arrayOne = a[0] ... a[n/2]
    arrayTwo = a[n/2+1] ... a[n]

    arrayOne = mergesort ( arrayOne )
    arrayTwo = mergesort ( arrayTwo )

    return merge ( arrayOne, arrayTwo )

merge ( array a, array b )
    array c

    while ( a and b have elements )
        if ( a[0] > b[0] )
            add b[0] to the end of c
            remove b[0] from b
        else
            add a[0] to the end of c
            remove a[0] from a

    // At this point either a or b is empty

    while ( a has elements )
        add a[0] to the end of c
        remove a[0] from a

    while ( b has elements )
        add b[0] to the end of c
        remove b[0] from b

    return c
```

$O(n \log n)$

<https://youtu.be/4VqmGXwpLqc?si=0eDmib5zhqgbQ0km>

Pseudocódigo MergeSort

A continuación se presenta el pseudocódigo de la implementación, basado en lo anterior:

```
mergesort (array a)
    if ( n == 1 )
        return a
    arrayOne = a[0] ... a[n/2]
    arrayTwo = a[n/2+1] ... a[n]
    arrayOne = mergesort ( arrayOne )
    arrayTwo = mergesort ( arrayTwo )
    return merge ( arrayOne, arrayTwo )

merge ( array a, array b )
    array c
    while ( a and b have elements )
        if ( a[0].tenure >= b[0].tenure )
            add a[0] to the end of c
            remove a[0] from a
        else
            add b[0] to the end of c
            remove b[0] from b
    // At this point either a or b is empty
    while ( a has elements )
        add a[0] to the end of c
        remove a[0] from a
    while ( b has elements )
        add b[0] to the end of c
        remove b[0] from b
    return c
```

El único cambio respecto a la imagen original es la condición $a[0].tenure \geq b[0].tenure$ en lugar de $a[0] > b[0]$, lo que produce un ordenamiento descendente por tenure en lugar de ascendente.

Pseudocódigo búsqueda binaria recursiva

binarySearch(arr, left, right, k):

si $\text{left} > \text{right}$: retornar -1

$\text{mid} = (\text{left} + \text{right}) / 2$

si $\text{arr}[\text{mid}].\text{tenure} == k$: retornar mid

si $\text{arr}[\text{mid}].\text{tenure} > k$:

retornar $\text{binarySearch}(\text{arr}, \text{mid}+1, \text{right}, k)$

sino:

retornar $\text{binarySearch}(\text{arr}, \text{left}, \text{mid}-1, k)$

Análisis de complejidad

MergeSort: La recurrencia es $T(n) = 2T(n/2) + O(n)$. Aplicando el Teorema Maestro con $a=2$, $b=2$, $f(n)=\Theta(n)$: como $f(n) = O(n^{\log_2 2}) = O(n)$, aplica el caso 2: $T(n) = O(n \log n)$. Es preferible a QuickSort para este problema porque garantiza este costo incluso en el peor caso (arreglo ya ordenado), y es estable, preservando el orden relativo de registros con el mismo tenure.

Búsqueda binaria

$T(n) = T(n/2) + O(1) \rightarrow O(\log n)$.

Tabla de tiempos empíricos

n	tiempo (ms)	$n \cdot \log_2(n)$	razón tiempo	razón $n \cdot \log_2 n$
1,000	0.11	9,966	1.00×	1.00×
3,500	0.42	41,196	3.82×	4.13×
7,043	0.85	90,009	7.73×	9.03×

Las razones entre tiempos son consistentes con el crecimiento teórico $O(n \log(n))$.

Resultados de las 5 consultas

Consulta	k	customerID encontrado
Q A01	72	3133-PZNSR
Q A02	60	3154-HMWUU
Q A03	45	7925-PNRGI
Q A04	30	6754-WKSHP
Q A05	12	6625-UTXEW

4. Módulo B - Codicioso

Pregunta 4.

Aristas del subgrafo

Arista	Cálculo	Costo
(0, 1)	$63.61 + 63.94 = 127.55$	127

(0, 2)	$63.61 + 64.37 = 127.98$	127
(0, 3)	$63.61 + 63.25 = 126.86$	126
(0, 4)	$63.61 + 63.63 = 127.24$	127
(1, 2)	$63.94 + 64.37 = 128.31$	128
(1, 3)	$63.94 + 63.25 = 127.19$	127
(1, 4)	$63.94 + 63.63 = 127.57$	127
(2, 3)	$64.37 + 63.25 = 127.62$	127
(2, 4)	$64.37 + 63.63 = 128.00$	128
(3, 4)	$63.25 + 63.63 = 126.88$	126

Ejecución manual de Kruskal

Paso	Arista	Costo	Componentes antes	¿Forma ciclo?	Acción
1	(0, 3)	126	{0}, {1}, {2}, {3}, {4}	No	Agregar
2	(3, 4)	126	{0,3}, {1}, {2}, {4}	No	Agregar
3	(0, 1)	127	{0,3,4}, {1}, {2}	No	Agregar
4	(0, 2)	127	{0,1,3,4}, {2}	No	Agregar

MST del subgrafo: **(0,3), (3,4), (0,1), (0,2)**

Peso total: $126 + 126 + 127 + 127 = 506$

Pseudocódigo.

CREAR-CONJUNTO(x):

 padre[x] $\leftarrow$ x

 rango[x] $\leftarrow$ 0

ENCONTRAR(x):

 si padre[x] $\neq$ x:

 padre[x] $\leftarrow$ ENCONTRAR(padre[x])

 retornar padre[x]

UNIR(x, y):

```

    rx ← ENCONTRAR(x)
    ry ← ENCONTRAR(y)
    si rx = ry:
        retornar falso
    si rango[rx] < rango[ry]:
        intercambiar(rx, ry)
    padre[ry] ← rx
    si rango[rx] = rango[ry]:
        rango[rx] ← rango[rx] + 1
    retornar verdadero
KRUSKAL(V, E):
    para cada v que pertenece V:
        CREAR-CONJUNTO(v)
    Ordenar E en orden no decreciente por peso
    MST ← ∅
    pesoTotal ← 0
    para cada (u, v, w) ∈ E (en orden de peso):
        si UNIR(u, v):
            MST ← MST ∪ {(u, v, w)}
            pesoTotal ← pesoTotal + w
        si |MST| = |V| - 1:
            terminar
    retornar MST, pesoTotal

```

Lema del ciclo: Para cualquier corte $(S, V - S)$ de un grafo conexo ponderado, la arista de menor peso que cruza el corte pertenece a algún MST.

Por ejemplo, tomando el corte $S = \{5\}$, $V - S = \{\text{Nodos sin } 5\}$. La arista $(5, 10)$ es la más barata que cruza este corte.

Suponiendo que existe un MST T^* que no la incluye. Entonces T^* usa otra arista más cara para conectar el nodo 5 al resto. Si se añade $(5, 10)$ a T^* se forma un ciclo, y al quitar esa arista más cara del ciclo obtenemos un árbol con peso menor que T^* — lo que es una contradicción.

Por lo tanto, (5, 10) debe estar en el MST. Kruskal aplica este razonamiento en cada paso: siempre elige la arista más barata que no forme un ciclo, garantizando el óptimo global.

Nodo u	Nodo v	Costo
5	10	123
0	5	124
3	5	124
4	5	124
2	5	125
10	14	125
1	10	125
5	11	125
5	9	125
5	19	126
5	17	126
5	12	126
10	15	126
5	13	126
5	16	127
5	6	127
5	7	127
5	18	128
5	8	129

Peso total del MST: 2,388

5. Módulo C - Programación Dinámica

La formulación de recurrencia de mochila 0-1 se modela el problema, donde cada solicitud puede ser seleccionada una sola vez o descartarla.

Cada una de las solicitudes i tiene: peso, valor, capacidad total, sería: $w_i = \text{round}(\text{TotalCharges})$, $v_i = \text{round}(\text{MonthlyCharges} * 10)$, $w = 500$.

Se define: $dp[i][w]$ eso es igual al valor máximo que se puede obtener usando las primeras i solicitudes con capacidad máxima w .

La recurrencia se presenta de esta manera:

Si $w_i > w$:

$$dp[i][w] = dp[i-1][w]$$

Si $w_i \leq w$:

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i)$$

El primer caso significa que la solicitud no cabe. El segundo caso compara dos opciones: no tomar la solicitud o tomarla y sumar su valor.

Condición inicial es:

$$dp[0][w] = 0$$

$$dp[i][0] = 0$$

esto es porque con cero solicitudes o capacidades cero no se puede obtener valor.

El **contraejemplo del enfoque codicioso** es, cuando aplicamos el enunciado sobre las primeras 50 solicitudes activas de mayor, se encontró que todas tienen peso mayor que 500, por lo que ninguna puede entrar en la mochila. Así que la solución óptima para esa instancia es valor 0.

Demostrar el fallo del enfoque codicioso, se construyó una mini instancia con exactamente 3 solicitudes reales del csv, teniendo en cuenta que se mantiene:

$$\text{Peso} = \text{round}(\text{TotalCharges})$$

$$\text{Valor} = \text{round}(\text{MonthlyCharges} * 10)$$

La capacidad de la mini instancia $W_{ce} = 38$

Solicitudes	Peso	Valor	Ratio v/w
0673-IGUQO	20	205	10.2500
2220-IAHLS	19	194	10.2105
5766-ZJYBB	19	194	10.2105

La tabla comparativa

Enfoque	Solicitudes seleccionadas	Valor total	¿óptimo?
Codicioso ratio v/w	0673-IGUQO	205	No
PD Mochila 0-1	2220-IAHLS, 5766-ZJYBB	388	Sí

Explicación de la tabla, el algoritmo codicioso selecciona primero la solicitud 0673-IGUQO, ya que tiene el mayor ratio. Tener en cuenta que al tomarla usa 20 unidades de capacidad y quedan solo 18, por lo que no puede agregar ninguna de las otras dos solicitudes. Su valor total queda en 205.

Mientras que la programación dinámica evalúa todas las combinaciones posibles y encuentra que seleccionar 2220-IAHLS y 5766-ZJYBB usa exactamente 38 unidades de

capacidad y obtiene valor 388, que es mayor. Así que el enfoque codicioso no garantiza optimalidad en Mochila 0-1.

La discusión de pseudopolinomialidad, la tabla dinámica tiene dimensiones:

$$(n + 1) \times (W + 1)$$

Así que, la complejidad temporal es: $\Theta(n \cdot W)$ y la complejidad espacial también es: $\Theta(n \cdot W)$, porque se almacena toda la tabla dp para luego reconstruir la solución mediante backtracking.

Aunque parece polinomial, este algoritmo es pseudopolinomial, porque depende del valor numérico de W, no solamente del tamaño de la entrada en bits. Es decir, si W crece mucho, la tabla también crece proporcionalmente, aunque el número de solicitudes n se mantenga igual.

Pseudocódigo de la tabulación

MOCHILA_01_TABULACION(items, W)

Entrada:

items: arreglo de n solicitudes

 cada solicitud i tiene:

 peso[i] = round(TotalCharges)

 valor[i] = round(MonthlyCharges * 10)

W: capacidad máxima de ancho de banda

Salida:

valor óptimo total

solicitudes seleccionadas

1. n <- cantidad de solicitudes

2. Crear tabla dp de tamaño (n + 1) x (W + 1)

 inicializada en 0

3. Para i desde 1 hasta n:

 Para w desde 0 hasta W:

 si peso[i - 1] > w entonces

 dp[i][w] <- dp[i - 1][w]

 si no

 no_tomar <- dp[i - 1][w]

```
tomar <- dp[i - 1][w - peso[i - 1]] + valor[i - 1]
```

```
dp[i][w] <- max(no_tomar, tomar)
```

4. valor_optimo <- dp[n][W]

5. Reconstruir solución:

```
seleccionadas <- lista vacía
```

```
w <- W
```

Para i desde n hasta 1:

```
si peso[i - 1] <= w
```

```
y dp[i][w] != dp[i - 1][w]
```

```
y dp[i][w] == dp[i - 1][w - peso[i - 1]] + valor[i - 1]
```

entonces

```
agregar solicitud i - 1 a seleccionadas
```

```
w <- w - peso[i - 1]
```

6. Invertir seleccionadas

7. Retornar:

```
valor_optimo
```

```
seleccionadas
```

Pseudocódigo del backtracking:

BACKTRACKING_MOCHILA(dp, items, W)

Entrada:

dp: tabla ya construida de tamaño $(n + 1) \times (W + 1)$

items: arreglo de n solicitudes

W: capacidad máxima

Salida:

lista de solicitudes seleccionadas

1. seleccionadas <- lista vacía

2. w <- W

3. n <- cantidad de items

4. Para i desde n hasta 1:

```

item <- items[i - 1]

si item.peso <= w

  y dp[i][w] != dp[i - 1][w]

  y dp[i][w] == dp[i - 1][w - item.peso] + item.valor

entonces

  agregar item a seleccionadas

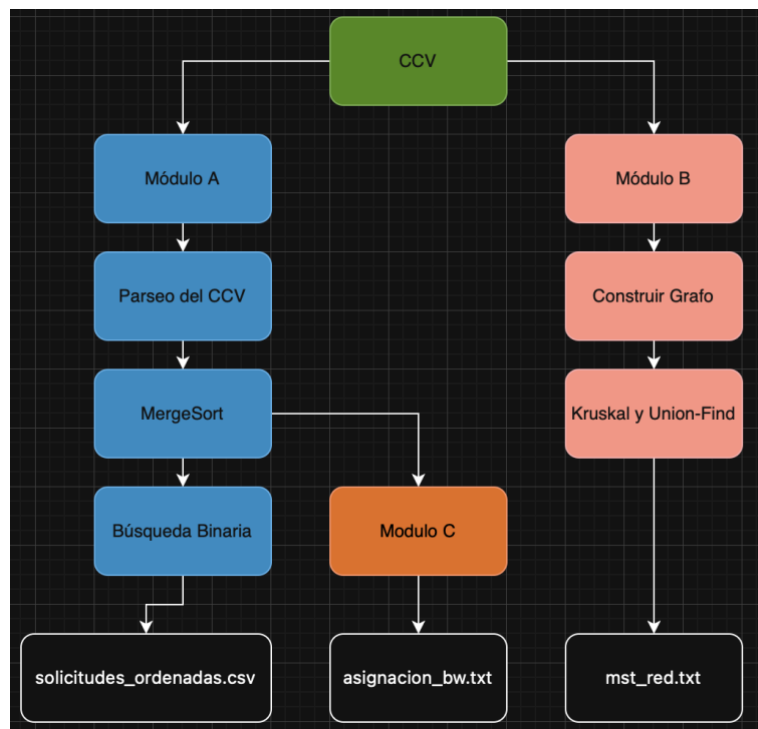
  w <- w - item.peso

```

5. Invertir seleccionadas

6. Retornar seleccionadas

6. Integración del pipeline



7. Conclusiones

MergeSort garantiza una complejidad de $O(n \log n)$ en el peor caso y es estable, lo cual lo hace ideal para ordenar las solicitudes sin alterar el orden relativo de registros con el mismo valor. Kruskal, por su parte, garantiza el óptimo global gracias a la elección codiciosa. Sin embargo, esta propiedad no siempre existe: como lo evidencia el Módulo C, el enfoque codicioso no produce la solución óptima en el problema de la Mochila 0-1, y fue necesaria la programación dinámica por tabulación para obtenerla.

A diferencia de instancias sintéticas donde los datos están balanceados y limpios, los datos reales presentaron poca variabilidad en MonthlyCharges entre grupos, lo que produjo un MST con topología de estrella doble en lugar de una estructura más distribuida; pesos desproporcionados en la mochila que limitan las soluciones factibles dentro de $W = 500$; y 11 registros con TotalCharges vacío que requirieron manejo especial en el parseo. Esto evidencia que los algoritmos funcionan correctamente en ambos escenarios, pero los datos reales producen estructuras menos diversas y exigen robustez para manejar casos más complejos.

8. Herramientas utilizadas

Se utilizó ChatGPT Modulo 5.5 versión Pro, Copilot plan gratuito como herramienta de apoyo para aclarar conceptos sobre Programación Dinámica, formulación de la recurrencia de Mochila 0-1, corrección de pseudopolinomialidad y comprensión del fallo del enfoque codicioso.

También, se utilizó Claude Sonnet 4.6 para resolver dudas puntuales surgidas durante el desarrollo de los módulos: en el Módulo A para aclarar el comportamiento de MergeSort y la lógica de la búsqueda binaria sobre arreglos ordenados; y en el Módulo B para entender la justificación formal del algoritmo de Kruskal y su correctitud al construir el árbol de expansión mínima.

El código final fue comprendido, adaptado y validado manualmente por nuestro equipo de trabajo.

9. Referencias

Dataset de Kaggle, material del curso, bibliografía consultada.

<https://youtu.be/2C2v2TT1fPc?si=3FvR5esVQJ3Z2KkE>

<https://medium.com/@fabiola.barajas20/soluci%C3%B3n-de-un-problema-tipo-mochila-utilizando-programaci%C3%B3n-din%C3%A1mica-52e0a637bff0>

<https://www.youtube.com/watch?v=NmG4g4Pizlw>

https://cp-algorithms.com/dynamic_programming/knapsack.html

<https://www.geeksforgeeks.org/dsa/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/>

<https://www.youtube.com/watch?v=71UQH7Pr9kU>

<https://youtu.be/4VqmGXwpLqc?si=0eDmib5zhqgbQ0km>

10. Roles del equipo

Contribución específica de cada integrante por módulo.

Módulo A: Juan Esteban Trujillo

Módulo B: Andrés Pérez

Módulo C: Athina Cappelletti